

Internal deployment service

Architecture review for a small engineering platform

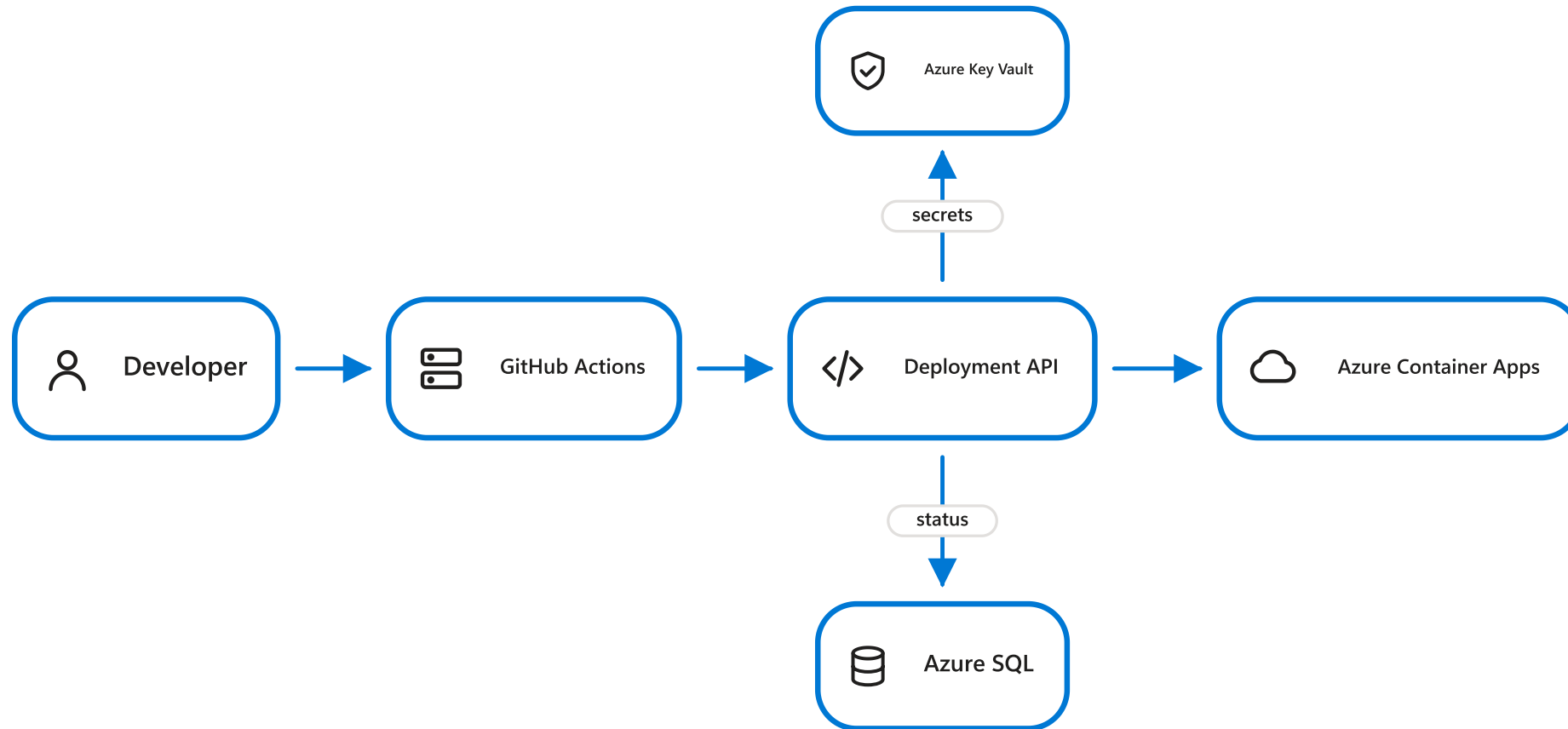
Make deployments safe and repeatable

- Teams need a **single, auditable path** from merge to production
- Manual deployment steps create drift, delays, and unclear ownership
- Objective: provide a small internal service that standardizes releases
- Optimize for low operational overhead and clear recovery paths

Guardrails for the first release

- Trigger deployments from **GitHub Actions** with authenticated requests
- Run workloads on **Azure Container Apps** with environment isolation
- Keep secrets in **Azure Key Vault**, never in workflow files or logs
- Persist release status and audit metadata in **Azure SQL**
- Support idempotent requests, observable failures, and operator rollback

A thin deployment control plane



Prove the path before expanding scope

- **Phase 1 — pilot:** one service, one Container Apps environment, least-privilege identities
- **Phase 2 — harden:** retries, idempotency keys, dashboards, and rollback runbooks
- **Phase 3 — scale:** onboard teams with templates, quotas, and environment policy
- Review deployment failure rate, lead time, and operator effort after each phase

Keep the control plane small

Risk	Mitigation
Credential misuse	Managed identity, scoped access, short-lived workflow tokens
Partial deployment failure	Idempotency, status transitions, and documented rollback
Service becomes a bottleneck	Queue long-running work and expose clear progress
Incomplete audit trail	Write immutable release events to Azure SQL

Next: confirm pilot owner, define the API contract, and select the first workload.

